Projeto N2 · Semana 2: Modelagem da camada de cache

1. Caminho escolhido: Redis (chave-valor) no padrão cache-aside

O trio confirma a escolha do Redis, já prevista na hipótese da Semana 1. O SQLite permanece como fonte da verdade, e o Redis armazena apenas o **resultado pronto** das consultas analíticas.

Caminho	Avaliação para o Painel ENEM 2025
Redis (escolhido)	O foco do projeto é desempenho. A base é somente leitura após a importação, então a mesma agregação sempre devolve o mesmo resultado. Cada resposta é pequena (27 linhas para "todas as UFs") e cabe em uma chave. TTL e DEL são nativos, e o servidor sobe no Docker com um comando.
MongoDB	Indicado para muitos registros com consultas variadas. Exigiria migrar 1.048.575 documentos, e o `$group` continuaria percorrendo a coleção inteira, trocando um *full scan* por outro.
Arquivo (JSON + Map)	Atenderia ao pré-cálculo das médias, mas não oferece TTL nem invalidação prontos e lida mal com combinações livres de UF e município.

Um índice no SQLite não resolveria o problema. Ele acelera o filtro por UF, mas as médias do Brasil e de todas as UFs continuam lendo o milhão de linhas. O custo está no **cálculo repetido**, e é esse cálculo que o cache elimina.

2. Unidade de dado e como ela é encontrada

A unidade de dado a ser persistida no Redis é o resultado estruturado das consultas analíticas que a interface gráfica consome. Em vez de armazenar registros individuais, a camada de cache guardará o conjunto consolidado de dados (ex.: a lista com os dados das 27 UFs ou as médias de um município específico) serializado no formato String com JSON, indexado por uma chave determinística com tempo de recuperação O(1).

Critério para inclusão no cache:

Para fundamentar a modelagem, foi realizado o diagnóstico de baseline na Semana 1 (com N = 1000 amostras sobre a base de 1.048.575 registros no Windows 11). A operação oficial de agregação analítica por estado registrou um tempo médio de 788,62 ms por execução, decorrente do *Full Table Scan* obrigatório na tabela sem índices.

Decisão sobre as consultas:

- **Estatísticas por UF (`GROUP BY`):** entra no cache (tempo médio de baseline: 788,62 ms, principal gargalo analítico).
- **Estatísticas sem filtro (Brasil consolidado + UFs):** entra no cache (envolve varredura nacional completa).
- **Estatísticas por Município / Município + UF:** entra no cache (custo de busca e agrupamento em texto sem índice).
- **Top 10 Redações:** entra no cache (custo de ordenação `ORDER BY` sobre a base inteira com `LIMIT`).
- **Consulta pontual por Candidato: não** entra no cache, pois a busca direta pela chave primária (`ID_RESULTADO`) já é indexada nativamente pelo SQLite e responde em tempo inferior a 1 ms.

Formato do valor em cache:

O resultado analítico será gravado como uma String contendo o array serializado das linhas da consulta, por exemplo:

```
[["AC", 6384, 480.75, 493.84, 513.99, 478.99, 555.01], ["AL", 21117, ...]]
```

Normalização das chaves:

Para garantir que filtros com acentuação ou espaços adicionais encontrem a chave correta no Redis sem duplicidade, os parâmetros de entrada passarão por uma função de normalização prévia (conversão para maiúsculas, remoção de acentos e substituição de espaços por sublinhados) antes da montagem da chave final.

Volume estimado de chaves:

Com 27 estados, cerca de 1.800 municípios e as consultas gerais e de ranking, o volume total previsto não ultrapassa 3.700 chaves no Redis (menos de 20 MB de consumo de memória RAM), permitindo configurar o container com um teto seguro de 64 MB de memória e política de descarte LRU (`allkeys-lru`).

3. Chave, TTL e invalidação

Cada combinação de filtros gera uma chave no padrão `enem:<recurso>:<parâmetros>`, com TTL padrão de 24 h. A invalidação combina as duas estratégias do enunciado: apagar as chaves na escrita e deixar o TTL como rede de segurança.

Chaves e TTL:

Tela / filtro	Chave	TTL
Estatísticas, sem filtro	`enem:stats:brasil` + `enem:stats:uf:todas`	86400 s
Estatísticas, UF = GO	`enem:stats:uf:GO`	86400 s
Estatísticas, município = Rio Verde	`enem:stats:mun:RIO_VERDE`	86400 s

Tela / filtro	Chave	TTL
Estatísticas, UF + município	`enem:stats:uf:GO:mun:RIO_VERDE`	86400 s
Top 10 (Brasil / UF)	`enem:top10:todas` · `enem:top10:uf:GO`	86400 s
Filtro sem resultado (ex.: UF `XX`)	a própria chave do filtro, com valor `null` ou `[]`	300 s
Índice de nomes de município (uso interno)	`enem:municipios:indice`	86400 s

A tela sem filtro usa duas chaves de propósito, lidas juntas com um `MGET`. Assim, `enem:stats:uf:todas` corresponde exatamente à consulta medida no baseline, e a comparação "antes × depois" mede a mesma operação.

Onde invalidar. Para atender ao critério de invalidação na escrita, será modelada na Semana 3 a operação **Corrigir Nota** (simulando recurso de prova), que atualizará a nota no SQLite e disparará o `DEL` das chaves impactadas no Redis.

Evento de escrita	1º passo: SQLite	2º passo: Redis
Correção da nota do candidato 123 (GO, Rio Verde)	leitura da UF e do município do candidato, seguida de `UPDATE` e `COMMIT`	`DEL enem:stats:brasil enem:stats:uf:todas enem:stats:uf:GO enem:stats:mun:RIO_VERDE enem:stats:uf:GO:mun:RIO_VERDE enem:top10:todas enem:top10:uf:GO`
Correção que altera UF ou município	igual	chaves da localidade **antiga e da nova**
Reimportação do CSV (`importar_sqlite.py`)	carga completa e `COMMIT`	remoção de todas as chaves `enem:*` com `SCAN` + `UNLINK` (sem `KEYS` nem `FLUSHALL`)
Nenhuma escrita	—	expiração automática pelo TTL de 24 h

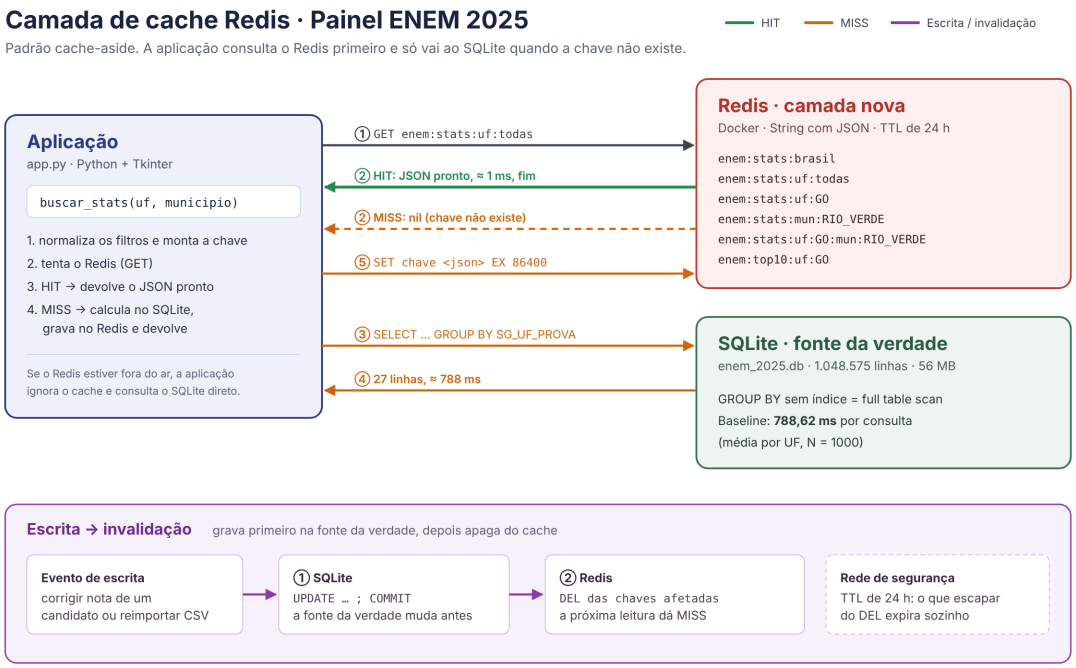
A regra para identificar as chaves afetadas: uma linha alterada muda **todas as agregações que a contêm**, ou seja, o Brasil, a lista de UFs, a UF do candidato, o município (sozinho e com a UF) e os dois rankings Top 10. As demais chaves continuam válidas.

Ordem das operações. A gravação acontece primeiro no banco e só depois a chave é apagada. Se o `DEL` viesse antes do `COMMIT`, uma leitura nesse intervalo daria MISS, leria o valor antigo no SQLite e o gravaria de volta no Redis por mais 24 h.

Falha do Redis. Se o Redis estiver fora do ar, a aplicação consulta o SQLite diretamente: a tela fica mais lenta, mas continua funcionando. Se um `DEL` falhar, o TTL limita o dado desatualizado a 24 h.

4. Diagrama: aplicação → Redis → SQLite

O diagrama arquitetural abaixo ilustra os fluxos operacionais previstos para a camada de persistência:



Detalhamento dos fluxos:

1. **Fluxo de Cache HIT (leitura rápida):**
    a. A aplicação Tkinter normaliza os filtros, gera a chave (ex.: `enem:stats:uf:todas`) e consulta o Redis via `GET`.
    b. O Redis localiza o registro em memória RAM e devolve o JSON pronto em cerca de 1 a 2 ms, finalizando a requisição sem acionar o disco.
2. **Fluxo de Cache MISS (primeira leitura ou chave expirada):**
    a. A aplicação realiza o `GET` no Redis e recebe retorno nulo (`nil`).
    b. A aplicação recorre à fonte da verdade (SQLite).
    c. O SQLite processa a agregação pesada em disco (operação que consome em média 788,62 ms no baseline medido na Semana 1).
    d. Os dados retornam para a aplicação.
    e. A aplicação grava o resultado no Redis via comando `SET` com parâmetro de expiração (`EX 86400` / 24 horas) e exibe os dados na tela.
3. **Fluxo de Escrita e Invalidação (garantia de consistência):**
    o Qualquer evento de escrita (como a funcionalidade planejada de correção de notas) efetuará a gravação na fonte da verdade primeiro (SQLite `UPDATE` + `COMMIT`).

- Imediatamente após a confirmação no banco relacional, a aplicação executará o comando `DEL` no Redis para as chaves correspondentes (localidade do participante e consolidados gerais), garantindo que a consulta seguinte dê MISS e recupere o dado atualizado.